Practical 12

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define NUM_ITEMS 100000

#ifndef BUFFER_SIZE
#define BUFFER_SIZE 5
#endif

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
   for (int i = 0; i < NUM_ITEMS; i++)
   {
```

```
        int item = i + 1;

        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 0; i < NUM_ITEMS; i++)
    {
        int item;

        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];
```

```
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    struct timespec start, end;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
```

```
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

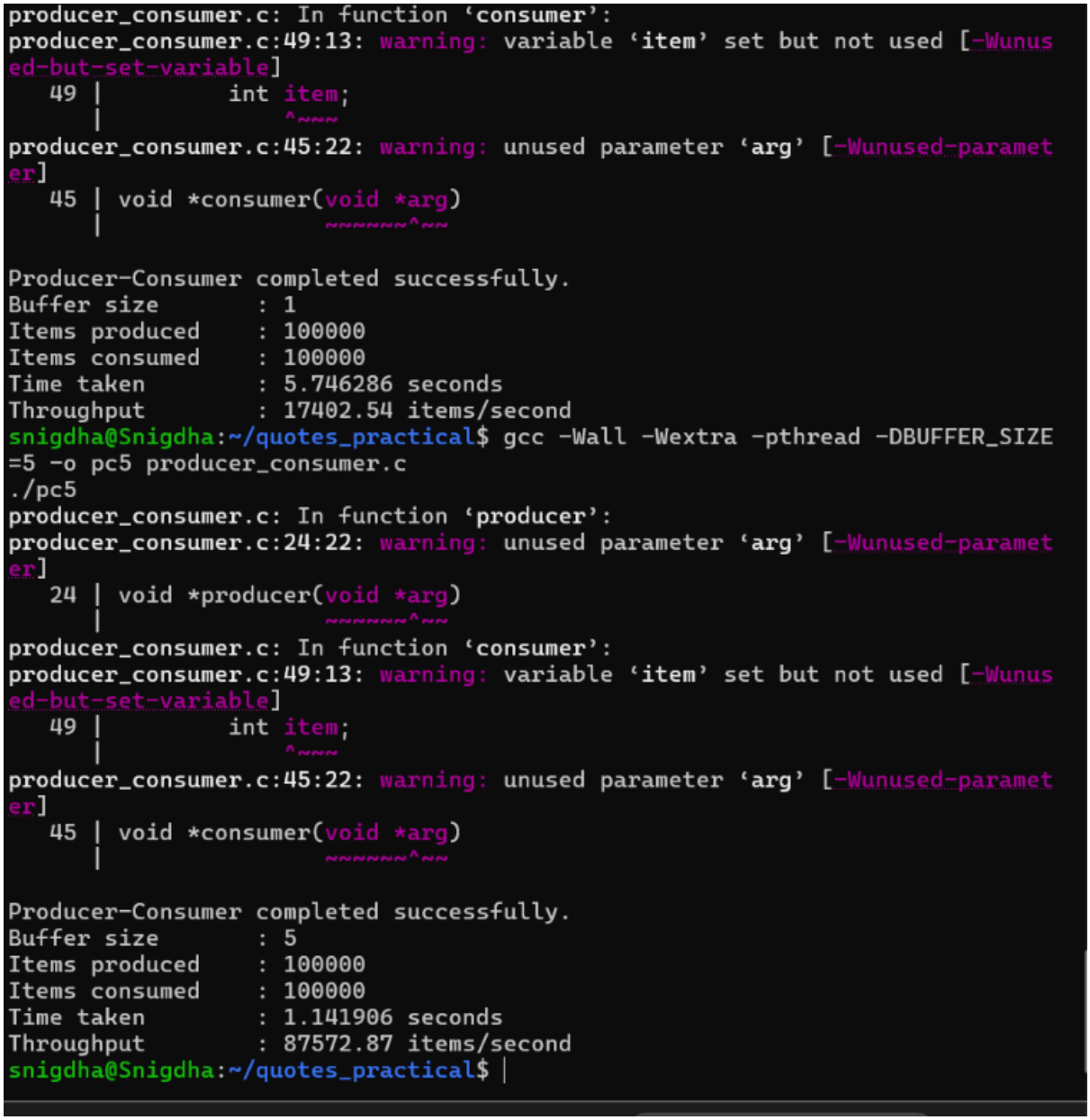
    printf("\nProducer-Consumer completed successfully.\n");
    printf("Buffer size      : %d\n", BUFFER_SIZE);
    printf("Items produced   : %d\n", NUM_ITEMS);
    printf("Items consumed   : %d\n", NUM_ITEMS);
    printf("Time taken       : %.6f seconds\n", time_taken);
    printf("Throughput       : %.2f items/second\n",
           NUM_ITEMS / time_taken);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ nano mutex_counter.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -pthread -o mutex_coun
ter mutex_counter.c
mutex_counter.c: In function ‘increment’:
mutex_counter.c:11:23: warning: unused parameter ‘arg’ [-Wunused-parameter]
   11 | void *increment(void *arg)
      |                 ~~~~~~^~~
snigdha@Snigdha:~/quotes_practical$ ./mutex_counter
Expected counter value: 4000000
Actual counter value: 4000000
snigdha@Snigdha:~/quotes_practical$ ./mutex_counter
Expected counter value: 4000000
Actual counter value: 4000000
snigdha@Snigdha:~/quotes_practical$ nano producer_consumer.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -pthread -o producer_c
onsumer producer_consumer.c
producer_consumer.c: In function ‘producer’:
producer_consumer.c:24:22: warning: unused parameter ‘arg’ [-Wunused-paramet
er]
   24 | void *producer(void *arg)
      |                ~~~~~~^~~
producer_consumer.c: In function ‘consumer’:
producer_consumer.c:49:13: warning: variable ‘item’ set but not used [-Wunus
ed-but-set-variable]
   49 |         int item;
      |             ^~~~
producer_consumer.c:45:22: warning: unused parameter ‘arg’ [-Wunused-paramet
er]
   45 | void *consumer(void *arg)
      |                ~~~~~~^~~
snigdha@Snigdha:~/quotes_practical$ ./producer_consumer

Producer-Consumer completed successfully.
Buffer size       : 5
Items produced    : 100000
Items consumed    : 100000
Time taken        : 1.120605 seconds
Throughput        : 89237.49 items/second
snigdha@Snigdha:~/quotes_practical$
```




```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>


pthread_mutex_t resource1;

pthread_mutex_t resource2;


void *thread1_function(void *arg)
```

```
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 locked\n");

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 locked\n");

    sleep(1);
```

```
    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed.\n");
```

```
    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ nano deadlock.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -pthread -o deadlock (
eadlock.c
deadlock.c: In function ‘thread1_function’:
deadlock.c:8:30: warning: unused parameter ‘arg’ [-Wunused-parameter]
    8 | void *thread1_function(void *arg)
      |                        ~~~~~~^~~
deadlock.c: In function ‘thread2_function’:
deadlock.c:28:30: warning: unused parameter ‘arg’ [-Wunused-parameter]
   28 | void *thread2_function(void *arg)
      |                        ~~~~~~^~~
snigdha@Snigdha:~/quotes_practical$ ./deadlock
Thread 1: locking Resource 1
Thread 1: Resource 1 locked
Thread 2: locking Resource 2
Thread 2: Resource 2 locked
Thread 1: waiting for Resource 2
Thread 2: waiting for Resource 1
```

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 locked\n");
```

```
    sleep(1);

    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked\n");

    printf("Thread 1: using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: resources released\n");

    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked\n");

    sleep(1);

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);
```

```
    printf("Thread 2: Resource 2 locked\n");

    printf("Thread 2: using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: resources released\n");

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);
```

```
    printf("\nNo deadlock occurred.\n");

    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ nano deadlock_prevention.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -pthread -o deadlock_p
revention deadlock_prevention.c
deadlock_prevention.c: In function ‘thread1_function’:
deadlock_prevention.c:8:30: warning: unused parameter ‘arg’ [-Wunused-parame
ter]
    8 | void *thread1_function(void *arg)
      |                        ~~~~~~^~~
deadlock_prevention.c: In function ‘thread2_function’:
deadlock_prevention.c:32:30: warning: unused parameter ‘arg’ [-Wunused-param
eter]
   32 | void *thread2_function(void *arg)
      |                        ~~~~~~^~~
snigdha@Snigdha:~/quotes_practical$ ./deadlock_prevention
Thread 1: locking Resource 1
Thread 1: Resource 1 locked
Thread 2: locking Resource 1
Thread 1: locking Resource 2
Thread 1: Resource 2 locked
Thread 1: using both resources
Thread 1: resources released
Thread 2: Resource 1 locked
Thread 2: locking Resource 2
Thread 2: Resource 2 locked
Thread 2: using both resources
Thread 2: resources released

No deadlock occurred.
```